

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1-Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	BHOOMIKA S	Reg.No.:	192524291
Department:		Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

Ans:

Objective

The objective is to construct an efficient I/O communication mechanism for a real-time embedded system where multiple sensors generate interrupts every 1 ms. The system must maintain CPU utilization below 15%, provide fast interrupt response, and use vectored interrupt handling.

Design and Construction

The proposed system consists of a CPU, multiple sensors, an interrupt controller, memory, and an I/O interface. Since the sensors generate interrupts frequently, a **vectored interrupt mechanism** is used. In vectored interrupts, every interrupt source is assigned a specific interrupt vector or address. When an interrupt occurs, the CPU can directly identify the corresponding service routine without checking all devices.

For I/O communication, **memory-mapped I/O** is selected because it provides fast access to device registers using normal memory instructions. This reduces the complexity of the instruction set and can provide low-latency communication, which is important in real-time systems.

Working

When a sensor generates an interrupt every 1 ms, the interrupt controller receives the request and identifies the source. It then provides the corresponding interrupt vector to the CPU. The CPU immediately executes the appropriate Interrupt Service Routine (ISR).

To keep CPU utilization below 15%, the ISR should be very short. It should only read the required sensor data, store it in a buffer, and clear the interrupt. More complex processing can be performed later as a background task. Interrupt priorities can also be assigned so that time-critical sensors are serviced first.

Conclusion

The proposed mechanism provides fast and predictable I/O communication using **vectored interrupts and memory-mapped I/O**. Keeping ISRs short and processing data using background tasks helps maintain CPU utilization below 15% while ensuring that sensor interrupts generated every 1 ms are handled efficiently.

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

Ans:

Objective

The objective is to design a high-speed I/O subsystem capable of achieving a storage throughput greater than **3 GB/s** using **NVMe over PCIe**, while minimizing CPU involvement in bulk data transfer.

Design and Construction

The system consists of a CPU, PCIe interface, NVMe controller, NVMe SSD, DMA controller, system memory, and an interrupt controller.

NVMe (Non-Volatile Memory Express) is selected because it is specifically designed for high-speed SSD communication and uses the PCIe bus. PCIe provides high bandwidth and low latency, making it suitable for achieving a throughput greater than 3 GB/s.

A **DMA controller** is integrated between the NVMe device and main memory. DMA allows data to be transferred directly between the storage device and memory without requiring the CPU to copy every byte or word.

Working

When a read or write operation is required, the CPU prepares the required command and DMA information. The NVMe controller communicates through PCIe and transfers the data directly to or from main memory using DMA.

After the transfer is completed, the DMA controller or NVMe controller generates an interrupt to notify the CPU. The CPU then handles the completion notification instead of managing the entire data transfer.

This significantly reduces CPU overhead and allows the CPU to perform other tasks while the storage transfer is in progress.

Conclusion

The combination of **NVMe, PCIe, DMA, and interrupt notification** provides a high-performance I/O subsystem. DMA removes the CPU from bulk data movement, while NVMe over PCIe provides the required throughput of more than 3 GB/s.

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

Ans:

Objective

The objective is to construct a communication architecture supporting **six USB peripherals and two high-speed Thunderbolt devices**. The architecture must prevent interrupt starvation, support both hardware and software interrupts, and provide fair bus arbitration.

Design and Construction

The system consists of a CPU, interrupt controller, USB host controller, six USB peripherals, Thunderbolt controller, two Thunderbolt devices, memory, and a bus arbitration mechanism.

The six USB peripherals are connected through a USB hub or host controller. The two high-speed devices are connected through a Thunderbolt controller. Since Thunderbolt devices can generate high-speed data traffic, their communication must be managed carefully so that USB devices are not completely blocked.

Working

When a device requires CPU attention, it generates an interrupt. **Hardware interrupts** are generated by physical devices such as USB and Thunderbolt peripherals. **Software interrupts** are generated by software when operating-system services or background operations are required.

An interrupt controller manages incoming interrupts and assigns priorities. High-priority interrupts are serviced quickly, but lower-priority devices must also receive service to prevent starvation.

For bus access, a **fair arbitration technique**, such as round-robin arbitration, can be used. Each device gets an opportunity to access the bus. After a device completes its transfer or reaches its allocated time/transfer limit, the next device receives bus access.

Conclusion

The proposed architecture allows six USB devices and two Thunderbolt devices to communicate efficiently. Interrupt prioritization prevents critical devices from experiencing excessive delays, while fair bus arbitration prevents interrupt or bus starvation. Hardware and software interrupts provide efficient communication between devices and the operating system.

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Ans:

Objective

The objective is to design a data acquisition system capable of continuously handling a data stream of **500 MB/s** with minimum CPU intervention. The system must compare DMA-based transfer with interrupt-driven I/O and use memory-mapped I/O for device registers.

Design and Construction

The proposed system contains a data acquisition device, CPU, DMA controller, main memory, and interrupt controller.

The data acquisition device continuously receives data from sensors or other sources at a rate of 500 MB/s. Since this is a high and continuous data rate, using interrupts for every data item would generate a very large number of interrupts and consume significant CPU time.

Therefore, **DMA is preferred** for transferring the data. The device registers are accessed using **memory-mapped I/O**, allowing the CPU to communicate with the acquisition device using normal memory access instructions.

DMA Transfer

In DMA-based operation, the CPU initially configures the DMA controller with the source address, destination address, transfer size, and control information. Once configured, the DMA controller transfers blocks of data directly from the acquisition device to main memory.

The CPU does not need to handle every data item. It can continue executing other tasks while DMA performs the transfer. After a block of data has been transferred, an interrupt can be generated to notify the CPU.

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Ans:

Objective

The objective is to construct an interrupt handling mechanism that supports **nested interrupts**, provides a maximum interrupt latency of less than **10 μ s**, uses vectored interrupts for high-priority devices, and uses software interrupts for background tasks.

Design and Construction

The system consists of a CPU, interrupt controller, high-priority devices, low-priority devices, interrupt vector table, and software interrupt mechanism.

Interrupts are assigned different priority levels. High-priority devices use **vectored interrupts**, allowing the CPU to directly identify the appropriate Interrupt Service Routine.

The system also supports **nested interrupts**, meaning that while the CPU is servicing a low-priority interrupt, a higher-priority interrupt can interrupt the current ISR.

Working

When a high-priority device generates an interrupt, the interrupt controller sends the request to the CPU. The CPU saves the current execution state and uses the interrupt vector to directly jump to the appropriate ISR.

If another higher-priority interrupt occurs while the current ISR is executing, the CPU temporarily suspends the current ISR and services the new interrupt. After the higher-priority ISR finishes, the CPU returns to the previously interrupted ISR.

To maintain the maximum latency below 10 μ s, the system should use fast interrupt recognition, vectored interrupt lookup, priority-based interrupt control, and short ISRs. Time-consuming processing should not be performed inside the ISR.

Software interrupts can be used for background tasks or operating-system services because they do not require an external hardware device to generate the interrupt.

Conclusion

The proposed interrupt mechanism provides fast and prioritized interrupt handling using **nested and vectored interrupts**. High-priority devices receive immediate attention, while software interrupts handle background operations. Keeping ISRs short and using efficient interrupt hardware helps maintain the required interrupt latency of less than 10 μ s.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided